Project overview, scope & development team

NexusGear — Technical Specification & Prototype Report | E-Commerce Project | Academic Year 2026–2027 | Milestone P2 (Week 7)

1.1 Executive summary & store niche

NexusGear is a custom-built, full-stack e-commerce web application engineered from scratch on ASP.NET Core (.NET 10 MVC) and SQLite. It connects enthusiast gaming peripherals with ergonomic workspace equipment in a streamlined, dark-themed buying journey.

The store serves three customer segments:

CUSTOMER SEGMENT	PRIMARY NEED	PRODUCT EXAMPLES
Competitive gamers	Precision and performance	39–49 g ultralight wireless mice; 8000 Hz polling; optical switches; compact hot-swappable keyboards.
Programmers & software engineers	Typing comfort and long-session ergonomics	Tactile typing acoustics; gasket-mounted aluminum chassis; split ergonomic layouts.
Remote professionals	Ergonomic seating and desk materials	Breathable mesh chairs with adaptive lumbar support; acoustic merino wool or glass deskmats.

The deterministic seeder (`DbSeeder.cs`) populates 30 original products across four categories: Mechanical Keyboards (8), Gaming Mice (8), Deskmats & Mousepads (8), and Ergonomic Gaming Chairs (6). Each seeded product includes custom studio imagery, technical description, price and stock level.

1.2 Technology stack summary

LAYER	TECHNOLOGY / IMPLEMENTATION
Frontend	HTML5, CSS3, modern JavaScript (ES6+) with Fetch API AJAX, Bootstrap 5.3 and Bootstrap Icons; Dark Tech design system; responsive to 360 px.
Backend	ASP.NET Core (.NET 10 MVC), layered monolith with Controllers, Services, Generic Repositories and Unit of Work wrapper.
Database	SQLite + Entity Framework Core 10; 3NF normalization across 11 tables; automated startup seeding.
External services	Stripe Checkout (Test Mode), verified server-side webhook `/api/webhook/stripe`, and Mailtrap SMTP for transactional HTML email.
Testing	`NexusGear.Tests` xUnit project; 53 tests passing at P2, using isolated In-Memory SQLite.

1.3 Team structure & ownership

TEAM MEMBER	ROLE	PRIMARY OWNERSHIP
Popa-Ion Andrei	Frontend Lead & Customer Experience	UI/UX, responsive layouts, catalog/category views, search/URL filters, recommendations UI, wishlist UI, checkout wizard and order history views (A1, A2, A5, A8, A11, A12, B1, B2, B7).
Șerban Alexandru-Cristian-George	Backend Lead & System Integrations	Layered architecture, 11-table SQLite/EF Core schema, Repository & Unit of Work, server cart/stock concurrency, BCrypt auth/address book, Stripe webhook, Admin portal and xUnit suite (A3, A4, A6, A7, A9, A10, A12, B3, B4, B5, B6).

Frozen specification — mandatory & bonus requirements

The P2 specification freezes the functional scope against which P3 delivery will be assessed.

2.1 Mandatory requirements A1–A12

- **A1 — Product catalog:** CatalogController + ProductRepository provide pagination (6 or 12 products/page, always ≤24), category views by unique Slug, and detail pages with imagery, specs, price and live stock; DbSeeder seeds 30 products.
- **A2 — Basic search:** Case-insensitive title/description search in ProductRepository; zero results show a clear empty state and a filter-reset action.
- **A3 — Shopping cart:** CartController + CartService add/update/remove items; server calculates subtotals/free-shipping threshold and rejects quantities above stock; guest Session items merge into IN_CART on login.
- **A4 — User accounts:** AccountController supports registration, cookie login and logout; passwords are BCrypt hashes; authenticated users manage saved addresses through M:N HAS_ADDRESS.
- **A5 — Checkout flow (P3 / W8):** Four-step wizard: address, delivery method (Standard Courier $15 / Express Next-Day $35), discount, server-calculated review and confirmation with OrderNumber NXG-2026-XXXX.
- **A6 — Sandbox payment (P3 / W9):** Create ORDER as Unpaid and initialize Stripe Checkout Test Mode; only a cryptographically verified checkout.session.completed webhook at /api/webhook/stripe changes status to Paid.
- **A7 — Stock management (P3 / W9):** Deduct stock in an atomic transaction on verified payment; EF Core optimistic concurrency via PRODUCT.RowVersion prevents overselling; cancellation restores inventory.
- **A8 — Order history (P3 / W10):** Authenticated OrdersController history shows date, number, status, shipping, discount and line items with snapshotted UnitPrice values.
- **A9 — Admin panel (P3 / W10):** Custom AdminController protected by Admin role; product/category CRUD, validated local image upload and controlled order state transitions.
- **A10 — Basic security:** ModelState validation, ValidateAntiForgeryToken on state-changing POSTs, parameterized EF Core LINQ, and server-side Admin role authorization.
- **A11 — Responsive design:** Mobile-first Bootstrap 5.3 layouts remain fully usable without horizontal overflow down to 360 px.
- **A12 — Automated tests:** 53 xUnit tests in NexusGear.Tests run with dotnet test against isolated In-Memory SQLite, covering catalog, cart, auth, wishlist, reviews and preview/error behavior.

2.2 Selected bonus features B1–B7

CODE / FEATURE	PTS	P2 STATUS	TECHNICAL IMPLEMENTATION
B1 Filtered Search	5	Implemented	Category, min/max price, in-stock filters; price/CreatedAt sorting; URL query preservation.
B2 Recommendations	4	Implemented	Frequently Bought Together from CONTAINS co-occurrence; Similar Products by category and price tier.
B3 Reviews & Ratings	4	Implemented	Verified-buyer gate; card averages; Admin IsHidden moderation.
B4 Sales Dashboard	3	Planned P3	Date filters, Chart.js daily revenue, Top 5 products and AOV.
B5 Discount Codes	3	Seeded / P3	NEXUS10 and SETUP50; ExpiryDate, MaxUses and UsedCount validation.
B6 Transactional Email	5	Planned P3	Mailtrap SMTP HTML receipt after verified Stripe webhook.
B7 Wishlist	2	Implemented	IN_WISHLIST persistence, navbar count and Move to Cart.

Conceptual data model — Chen ERD

Six core entities, four 1:N relationships and five associative M:N relationships.

3.1 Core entities and 1:N relationships

Core entities: CATEGORY, PRODUCT, USER, ADDRESS, ORDER and DISCOUNT_CODE. The four one-to-many relationships are shown below.

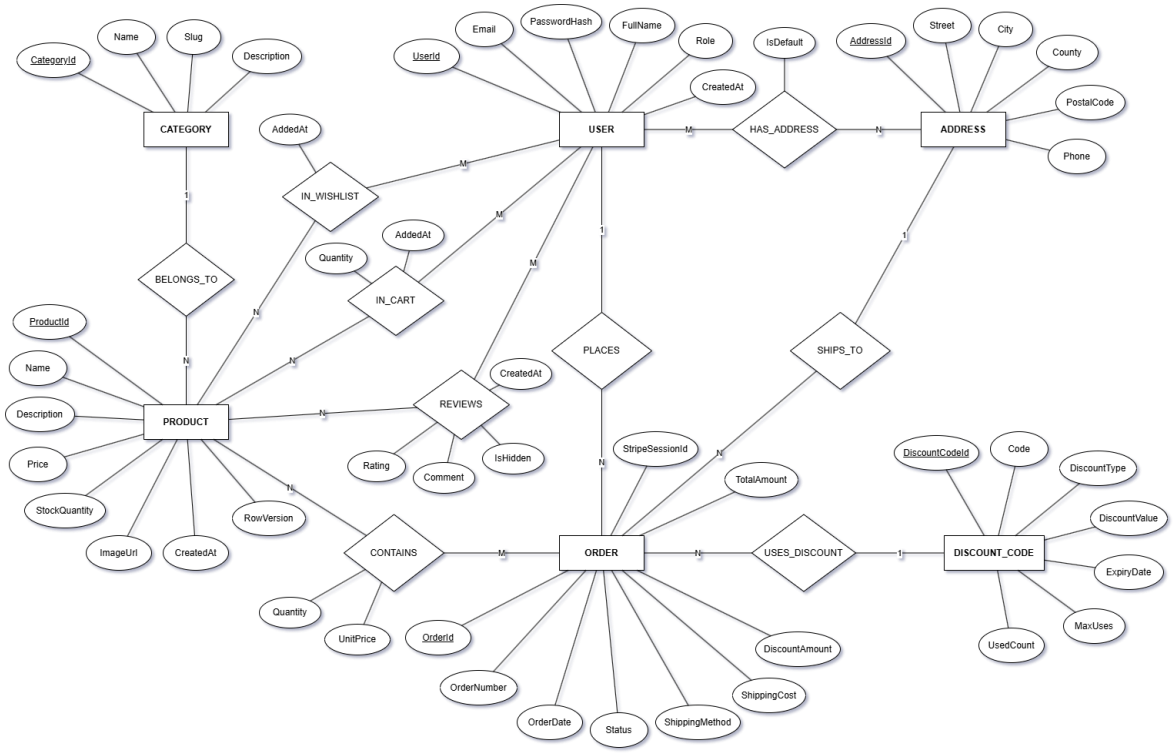
RELATIONSHIP	CARDINALITY	MEANING
BELONGS_TO	CATEGORY 1 : N PRODUCT	A category groups multiple products; each product belongs to a category.
PLACES	USER 1 : N ORDER	A user can place multiple orders; each order has one owning user.
SHIPS_TO	ADDRESS 1 : N ORDER	One physical address can be referenced by many orders; each order records its delivery destination.
USES_DISCOUNT	DISCOUNT_CODE 1 : N ORDER	A discount code may be used by multiple orders; an order may reference a discount code (nullable).

3.2 Associative M:N relationships

RELATIONSHIP	ENTITIES	RELATIONSHIP ATTRIBUTES / PURPOSE
HAS_ADDRESS	USER M:N ADDRESS	Allows multiple users to share one physical ADDRESS record; IsDefault belongs to this relationship so each user can have an independent default.
IN_CART	USER M:N PRODUCT	Persistent authenticated cart items; Quantity and AddedAt.
IN_WISHLIST	USER M:N PRODUCT	Saved products per user; AddedAt.
REVIEWS	USER M:N PRODUCT	Verified reviews; Rating, Comment, IsHidden and CreatedAt.
CONTAINS	ORDER M:N PRODUCT	Order line items; Quantity and historical UnitPrice snapshot.

MODEL NOTE HAS_ADDRESS stores IsDefault on the associative relationship, not on ADDRESS. The junction table names are preserved exactly in the physical relational model.

Figure 1. Conceptual Chen ERD: one-to-many and associative many-to-many relationships.



Physical relational model — 11 tables in 3NF

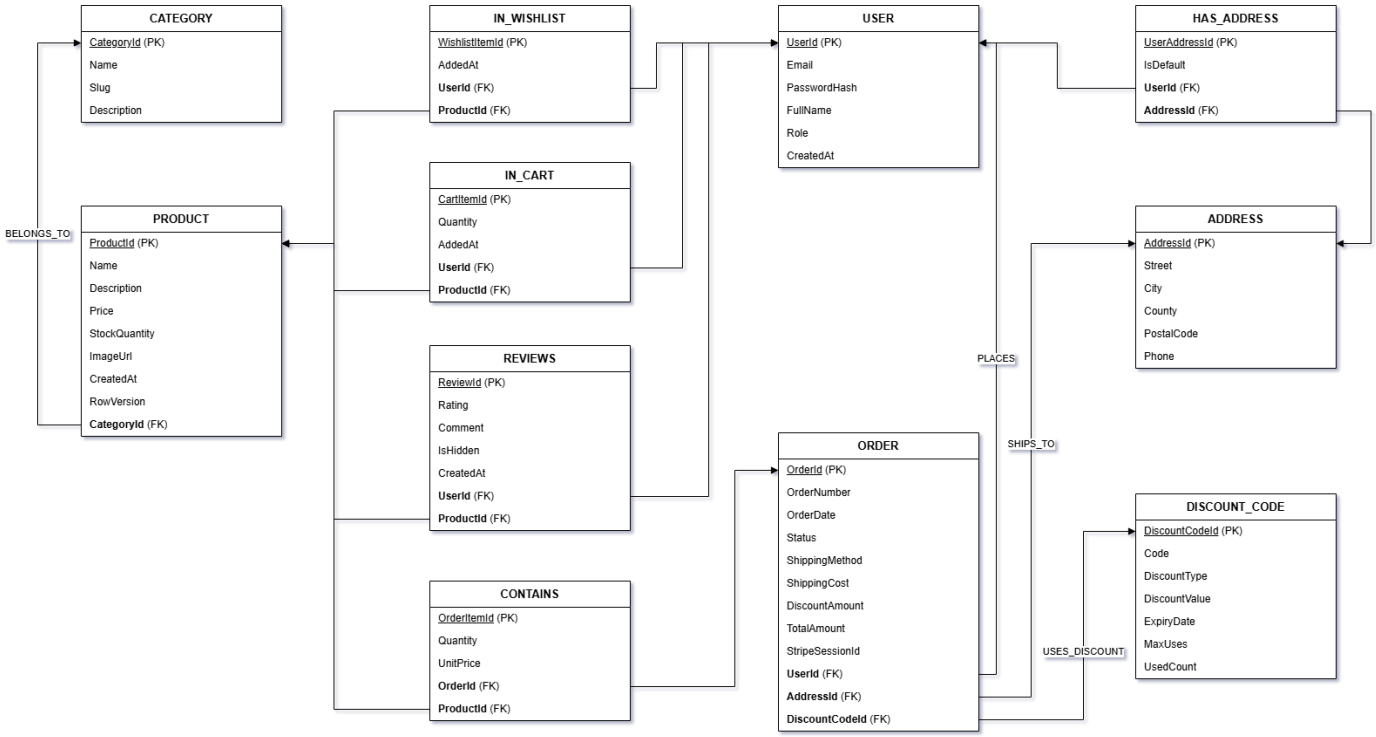
Every associative Chen relationship is materialized as a junction table with a matching name.

The SQLite relational model contains 11 tables in Third Normal Form and 14 foreign-key constraints configured through EF Core Fluent API in AppDbContext.cs.

4.1 Integrity and concurrency

- 14 FK constraints are enforced through EF Core Fluent API in AppDbContext.cs.
- CONTAINS stores UnitPrice at checkout time, isolating historical order totals from later catalog price changes.
- PRODUCT.RowVersion uses [ConcurrencyCheck] to guard simultaneous inventory updates (example seeded item: NexusTitan K-75, StockQuantity = 1).

Figure 2. Physical relational model overview: six core tables and five junction tables.



Architecture, design patterns & API endpoint catalogue

Layered monolithic ASP.NET Core (.NET 10 MVC) application; all authorization is enforced on the server.

5.1 Layered architecture and design patterns

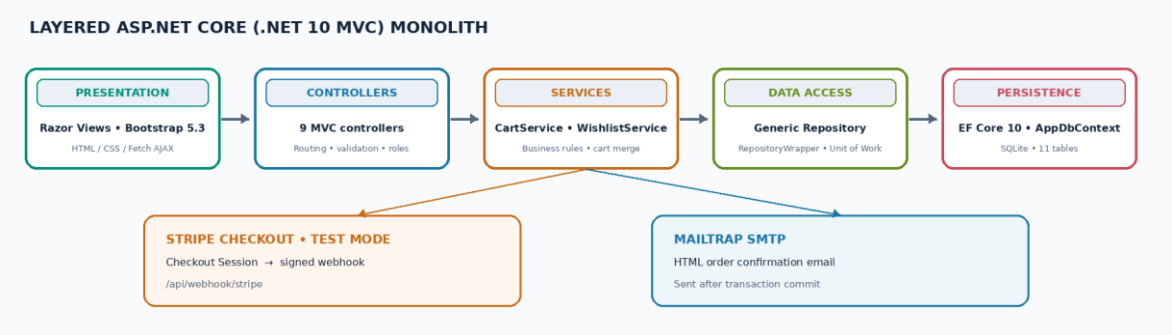
LAYER	PROJECT ELEMENTS	RESPONSIBILITY
Presentation	Views/ + wwwroot/; Razor .cshtml; partials _CartItemRow, _FreeShippingBar, _OrderSummary, _WishlistItemCard; Bootstrap 5.3; Fetch API AJAX	Render customer/admin UI and send interactive requests.
Controllers	Catalog, Cart, Wishlist, Account, Reviews, Checkout, Orders, Admin, Home	HTTP routing, ModelState validation, antiforgery and role checks.
Services	CartService, WishlistService; OrderService and EmailService in Stage 3	Business rules, guest-session-to-DB cart merge and stock boundaries.
Data access	Repositories/ + Data/; RepositoryBase<T>; IRepositoryWrapper / RepositoryWrapper; AppDbContext	Generic/domain repositories coordinated by Unit of Work.
External services	Stripe Checkout Test Mode + webhooks; Mailtrap SMTP	Sandbox payments, verified callbacks and transactional HTML email.

Generic Repository Pattern provides FindAll, FindByCondition, Create, Update and Delete operations; domain repositories include ProductRepository, CategoryRepository, CartRepository and WishlistRepository. Unit of Work coordinates repositories over one scoped AppDbContext and persists through SaveAsync(). Dependency Injection registers the application services in Program.cs.

5.2 API route and access catalogue

METHOD	ROUTE	CONTROLLER	ACCESS	FUNCTION
GET	/	HomeController.Index	Public	Storefront hero and categories
GET	/Catalog	CatalogController.Index	Public	Pagination, URL filters and sorting (A1/A2/B1)
GET	/Catalog/Details/{id}	CatalogController.Details	Public	Details, B2 recommendations and B3 reviews
GET	/Cart	CartController.Index	Public / Customer	Session or IN_CART cart (A3)
POST	/Cart/Add; /Cart/UpdateQuantity; /Cart/Remove	CartController	Public / Customer	Stock validation and server totals (A3)
GET/POST	/Wishlist; /Wishlist/Toggle; /Wishlist/MoveToCart	WishlistController	Authenticated Customer	Wishlist operations (B7)
GET/POST	/Account/Register; /Account/Login	AccountController	Public	BCrypt, cookie auth and guest-cart merge (A3/A4)
POST	/Account/Logout	AccountController.Logout	Customer / Admin	Cookie logout + CSRF (A4/A10)
GET/POST	/Account/Profile; /Account/AddAddress; /Account/SetDefaultAddress; /Account/DeleteAddress	AccountController	Authenticated Customer	HAS_ADDRESS management (A4)
POST	/Reviews/Create	ReviewsController.Create	Verified Buyer	Purchase verification (B3)
POST	/Reviews/ToggleVisibility	ReviewsController.ToggleVisibility	Admin role	Review moderation (A9/B3)
GET/POST	/Checkout; /Checkout/PlaceOrder	CheckoutController	Authenticated Customer	Checkout, discount validation and Stripe session (A5/A6/B5)
POST	/api/webhook/stripe	WebhookController.StripeWebhook	Stripe HMAC signature	Verifies event, deducts stock and marks Paid (A6/A7/B6)
GET	/Orders; /Orders/Details/{id}	OrdersController	Authenticated Customer	Personal order history (A8)
GET/POST	/Admin; /Admin/Products/*; /Admin/Orders/*; /Admin/Dashboard	AdminController	Admin role	CRUD, uploads, status, discounts and dashboard (A9/A10/B4/B5)

Figure 3. Layered architecture and external sandbox integrations.



Critical payment processing & concurrency-safe inventory

Transactional sequence: Checkout → Stripe sandbox → verified webhook → atomic stock update → Mailtrap SMTP.

Order fulfillment is decoupled from browser redirects. The SuccessUrl never marks an order Paid; only a cryptographically verified server-to-server webhook can trigger payment confirmation and inventory deduction.

6.1 End-to-end sequence

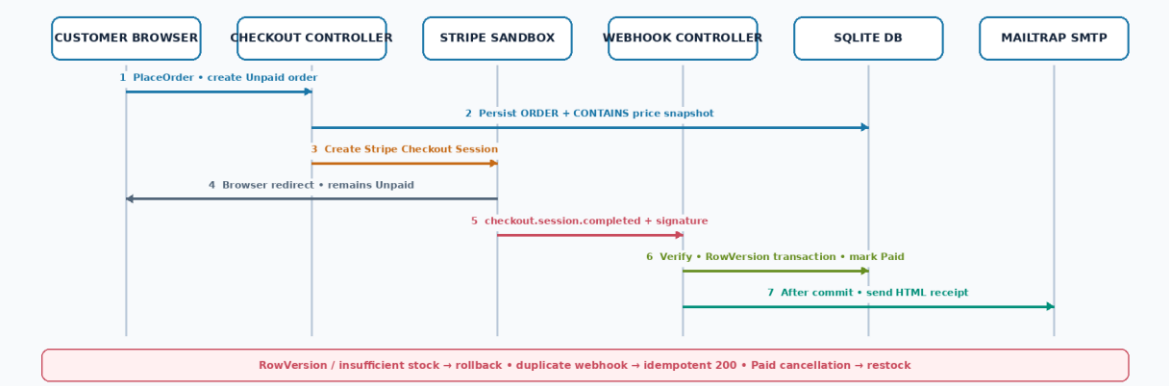
- **1 — Order creation (Unpaid):** Customer submits POST /Checkout/PlaceOrder. Server loads IN_CART, recalculates subtotal, validates discount expiry and usage, verifies stock, creates ORDER with Status = Unpaid, and snapshots each line UnitPrice in CONTAINS.
- **2 — Stripe Checkout Session:** Stripe.net in Test Mode creates a Checkout Session containing line items, shipping and discount; ClientReferenceId stores the order identifier. StripeSessionId is saved on ORDER and the browser is redirected to hosted checkout (HTTP 303).
- **3 — Sandbox payment & browser redirect:** Customer completes a Stripe test-card payment and is redirected to /Checkout/Confirmation/{orderId}. The order remains Unpaid while the database awaits the webhook.
- **4 — Webhook reception and signature verification:** Stripe posts checkout.session.completed to POST /api/webhook/stripe. WebhookController verifies the Stripe-Signature header against Stripe__WebhookSecret using EventUtility.ConstructEvent(); invalid signatures are rejected with HTTP 400.
- **5 — Atomic inventory deduction and concurrency:** On a valid event, the service begins a database transaction. If the order is already Paid, it returns HTTP 200 (idempotency). For each CONTAINS line, it checks StockQuantity ≥ Quantity, decrements stock and updates PRODUCT.RowVersion. It sets ORDER.Status = Paid, increments DiscountCode.UsedCount if applicable, clears IN_CART, saves changes and commits.
- **6 — Rollback, cancellation restock and email:** If a concurrent purchase consumed the last unit, DbUpdateConcurrencyException (or insufficient stock) causes transaction rollback and the second order is marked Cancelled. Admin cancellation of a Paid order restores quantities. After commit, EmailService sends an HTML confirmation receipt through Mailtrap SMTP.

6.2 Sequence participants

PARTICIPANT	RESPONSIBILITY
Customer Browser	Starts checkout, completes hosted test payment, receives confirmation redirect.
CheckoutController	Validates cart/discount/address; creates Unpaid order and Stripe session.
Stripe Sandbox	Processes test payment and dispatches checkout.session.completed.
WebhookController	Validates Stripe signature, handles idempotency and invokes payment/stock service.
SQLite DB	Stores order/line items; transaction applies RowVersion-guarded stock change, coupon count and cart cleanup.
Mailtrap SMTP	Receives transactional HTML confirmation after the database transaction commits.

LOCAL TESTING Forward events with: stripe listen --forward-to localhost:5000/api/webhook/stripe

Figure 4. Critical payment sequence from Unpaid order to webhook-confirmed stock update and email.



User interface design & wireframe specifications

Dark Tech UI system • Bootstrap 5.3 • responsive customer-facing views down to 360 px (A11).

Design tokens: Deep Obsidian #0B1120 background; Slate Navy #131D31 cards with #1E293B borders; Hero Mint #00E5A3; Electric Blue #38BDF8; white #FFFFFF and muted #94A3B8 typography.

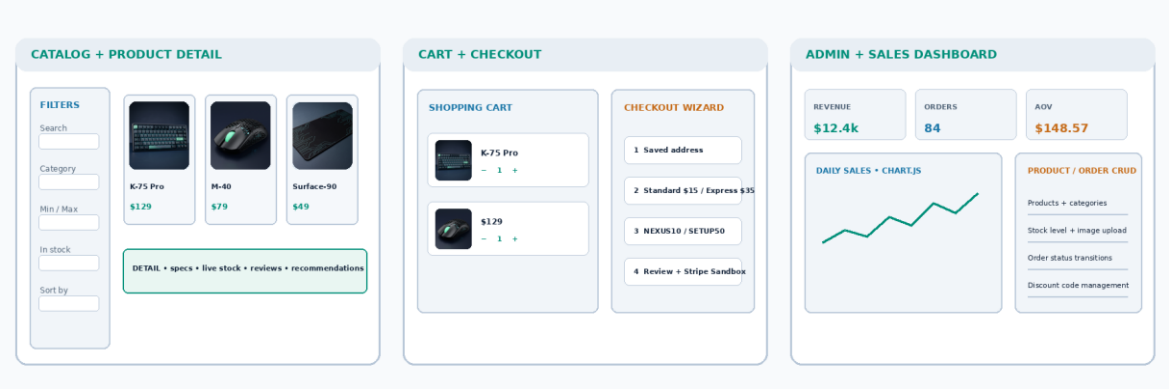
7.1 Five core screens

SCREEN / ROUTE	LAYOUT AND KEY ELEMENTS	STATUS / CODES
1. Product Catalog /Catalog	Desktop two-column layout: left filter sidebar and right product grid; collapses to single column with toggleable filter drawer at 360 px. Search + clear, category, min/max price, in-stock toggle, sort, URL-preserved pagination; product cards show 1:1 imagery, ratings, stock badges, wishlist heart and AJAX Add to Cart.	Implemented live A1, A2, A3, B1, B3, B7
2. Product Detail /Catalog/Details/{id}	Split hero: studio image on left; category, title, rating, price, live stock, technical description, quantity selector, Add to Cart and Wishlist on right. Lower sections show Frequently Bought Together, Similar Products and verified reviews with Admin Hide/Show controls.	Implemented live A1, A3, B2, B3, B7
3. Shopping Cart /Cart	Free Shipping Progress Bar toward $500; line-item rows with +/− AJAX steppers capped at StockQuantity; immediate remove; sticky order summary with server-calculated subtotal, shipping and total.	Implemented live A3
4. Checkout Wizard /Checkout	Four-step flow: saved HAS_ADDRESS selection (or new address); Standard $15 / Express $35 radio cards; NEXUS10 / SETUP50 discount; server summary and Pay with Stripe Sandbox CTA.	Stage 3 execution A5, A6, B5
5. Admin & Dashboard /Admin	Tabs for Sales Dashboard (date filter, Revenue/Orders/AOV KPI cards, Chart.js daily revenue, Top 5 products), Product/Category CRUD and image upload, order status management and discount code CRUD.	Stage 3 execution A9, B4, B5

7.2 UI implementation notes

- Shared partials: _CartItemRow.cshtml, _FreeShippingBar.cshtml, _OrderSummary.cshtml and _WishlistItemCard.cshtml.
- AJAX interactions use the Fetch API; server-side validation remains authoritative for stock, totals, discounts and access control.
- The listed layouts are wireframe specifications; screenshots of the live application should be inserted where available for the P2 presentation.

Figure 5. Wireframe previews: catalog/product detail, cart/checkout and admin dashboard.



Functional prototype report, test suite & local execution

P2 prototype status and evidence at the P2 milestone.

8.1 Functional prototype status

The delivered prototype is an ASP.NET Core (.NET 10) application that initializes its SQLite database on startup. Stage 2 demonstration covers catalog data from the database and user login; additional Stage 3 routes are connected to controllers and show a Stage 3 Roadmap Preview instead of returning HTTP 404.

- DbSeeder.cs seeds 4 categories, 30 products with local images Product_1.png–Product_30.png, 2 BCrypt-hashed accounts, a default HAS_ADDRESS record, discount codes NEXUS10 / SETUP50 / EXPIRED20, two historical orders and three product reviews.
- Seeded local test accounts: admin@nexusgear.ro / Admin123! and customer@nexusgear.ro / Customer123!. These are local demo credentials, not production credentials.
- Live prototype flow: start application → browse categories and products → use URL-preserved filters and empty-search feedback → add as guest → log in → verify Session-to-SQLite cart merge → inspect profile address book → move wishlist item to cart → submit verified review.

8.2 Automated test suite — 53 passing xUnit tests

TEST SUITE	COUNT	COVERAGE	RESULT
CatalogTests.cs	14	Pagination (≤24), category Slug, search/empty state, B1 filters/sorting, B2 recommendations.	14 / 14
CartTests.cs	13	Add/update/remove, server subtotal, free-shipping math, stock rejection, guest cart merge.	13 / 13
AccountAndReviewTests.cs	12	BCrypt verification, M:N address book/default address, verified-buyer review, Admin moderation.	12 / 12
WishlistTests.cs	8	Wishlist add/remove, duplicate prevention and Move to Cart.	8 / 8
PreviewAndErrorTests.cs	6	Admin authorization, Stage 3 preview routes, custom 404/500 handling.	6 / 6
TOTAL	53	Single-command test execution: dotnet test NexusGear.slnx	53 / 53

8.3 Local execution (five commands) & AI declaration

```
git clone https://github.com/alexandrucristiangeorgeserban/nexusgear.git
cd nexusgear
cp .env.example .env
dotnet test NexusGear.slnx
dotnet run --project NexusGear
```

AI tools declaration: Google Gemini, OpenAI ChatGPT and Antigravity IDE were used for specification consistency analysis, product studio illustration generation, boilerplate scaffolding and documentation formatting.

P2 DELIVERABLES Slides, technical specification, working catalog/login prototype and repository commit tagged P2.